第 1 題英文題目

1. True or False (20 pts) (note that the score will be decreased by one if the answer is not correct).a.

The advantages of data encapsulations includes 1) testing and debugging, 2) reusability, and 3) modifications to the representation of a data type.a.

資料封裝（data encapsulation）的優點包括：

(1) 測試與除錯

(2) 可重用性

(3) 當資料型別的表示法改變時，程式較容易修改

a.

True

這題是正確的。資料封裝（encapsulation）的核心概念是將資料（data）與操作這些資料的函式（methods）封裝在同一個類別中，並透過存取控制（如 private、public）限制外部直接操作內部資料。這樣的設計帶來多項優點。首先，在**測試與除錯**方面，由於資料操作集中在特定函式中，當錯誤發生時可以快速定位問題來源，而不需要在整個程式中搜尋，大幅提升除錯效率。其次，在**可重用性（reusability）**方面，**封裝良好的類別可以在不同程式或模組中重複使用，減少重複撰寫程式碼，並提升開發效率與一致性。最後，在**資料表示修改（modification of representation）**方面，若內部資料結構需要改變（例如從陣列改為鏈結串列），只要對外提供的介面不變，使用該類別的程式碼幾乎不需修改，這種特性稱為資訊隱藏（information hiding）。因此，題目所列三項皆為資料封裝的重要優點，本題為 True。

b.

We say that a data object of Type B IS-A data object of Type A if A is more specified than B.

b.

若 A 比 B 更具體（more specified），則 Type B 是 Type A（B IS-A A）。

b.

False

這題是錯的，因為 IS-A 關係方向顛倒。在物件導向中，IS-A 表示「子類別是父類別的一種」，也就是較具體的類別指向較抽象的類別。例如 Dog is an Animal，Dog 比 Animal 更具體，因此 Dog IS-A Animal。題目卻說「如果 A 比 B 更具體，則 B IS-A A」，這邏輯是反過來的。正確應該是「若 B 比 A 更具體，則 B IS-A A」。因此此敘述錯誤。

c.

A derived class can inherits all the non-private members (data and functions) of the base class including constructor and destructor.

c.

衍生類別可以繼承基底類別所有非 private 成員（資料與函式），包括建構子與解構子。

c.

False

這題錯在最後一句「including constructor and destructor」。在 C++ 中，衍生類別確實可以繼承基底類別的非 private 成員（如 public、protected 成員），但是建構子與解構子並不會被繼承。建構子是在物件建立時自動呼叫父類別的建構子來初始化父類別部分，解構子則在物件銷毀時依序呼叫，但這並不等同於繼承。因此題目敘述錯誤。

d.

Container classes are suitable for implementation using templates because the algorithms for basic container class operations are independent of the object type that the container class contains.

d.

容器類別適合使用樣板（templates）實作，因為基本操作的演算法與容器內物件型別無關。

d.

True

這題是正確的。容器類別如 stack、queue、list、vector 等，其基本操作（例如插入、刪除、存取）在邏輯上與元素的型別無關。也就是說，不論容器中存放的是整數、浮點數或物件，其操作流程都是相同的。因此可以使用 C++ 的 template 技術撰寫一次程式碼，並套用到不同型別上，提高程式的通用性與重用性。這正是樣板設計的核心目的，因此此敘述正確。

e.

Time complexity of a program is given by the number of steps taken by the program. Several statements can be considered as a computation unit if they are independent of characteristics of instance.

e.

程式的時間複雜度由執行步驟數決定，若某些敘述與輸入特性無關，則可視為一個計算單位。

e.

True

這題是正確的。時間複雜度分析的核心在於計算程式執行的基本操作次數，而

不是實際時間。通常我們會將某些基本運算（如加減乘除、比較、指派）視為一個計算單位，前提是這些操作不會因輸入資料的不同而改變其執行時間。因此，只要這些敘述與輸入特性無關，就可以當作常數時間操作來分析整體複雜度，這也是 Big-O 分析的基本概念。

f.

C++ allows a programmer to overload operations of user-defined data types by providing a definition that implements this operator. This definition must take the form of a class member function.

f.

C++ 允許對使用者自訂型別進行運算子多載，但此定義必須是類別成員函式。

f.

False

這題錯在「must take the form of a class member function」。雖然 C++ 確實允許對運算子進行多載，但這些運算子函式不一定要寫成類別成員函式，也可以寫成非成員函式，甚至常搭配 friend function 使用。例如輸出運算子 << 通常就是非成員函式。因此題目中的「必須是成員函式」這個限制是錯誤的，所以整題為 False。

g. $f(n) = \Omega(g(n))$ iff there exist positive constants c and n_0 such that $f(n) > cg(n)$ for all n if $n \geq n_0$.

g.

g. 若存在正常數 c 與 n_0，使得當 $n \geq n_0$ 時，$f(n) > cg(n)$，則且唯若 $f(n) = \Omega(g(n))$。

g. True

這題是在考 **Big-Omega 的正式定義**。

若 $f(n) = \Omega(g(n))$，表示從某個 n_0 開始，$f(n)$ 至少和 $g(n)$ 同階下界，也就是存在正的常數 c 與 n_0，使得對所有 $n \geq n_0$，都有

$$f(n) \geq cg(n)$$

題目寫成 $f(n) > cg(n)$，雖然嚴格來說課本常寫 $\geq$，但在漸近表示法中，這樣的寫法通常仍被接受為同義的下界概念，因此這題應判 **True**。考試中若老師照定義寬鬆判斷，這題就是正確。

h.

We have to double a circular queue when Front == Rear.

h.

當 circular queue 中 Front == Rear 時，我們必須將容量加倍。

h.

False

這題錯在條件判斷過於絕對。在 circular queue 中，Front == Rear 可能代表

「空」也可能代表「滿」，取決於實作方式。有些設計會保留一個空格來區分滿與空，也有些會額外使用 size 變數。因此不能單純依據 Front == Rear 就決定要擴充容量，所以此敘述錯誤。

i.

We can evaluate an expression through the queue data structure.

i.

我們可以透過 queue 資料結構來計算運算式。

i.

False

這題是錯的。運算式求值（例如 infix 轉 postfix 或 postfix evaluation）主要使用的是 stack，而不是 queue。因為 stack 具有後進先出（LIFO）特性，可以正確處理運算子優先順序與括號結構，而 queue 為先進先出（FIFO），不適合這類問題。因此此敘述錯誤。

j. The time complexity when we add two polynomials p_1 and p_2 is $O(m+n)$ if we increase the capa of an array by 3 each time it is full, where the number of terms in p_1 and p_2 are m and n, respectivel

j.

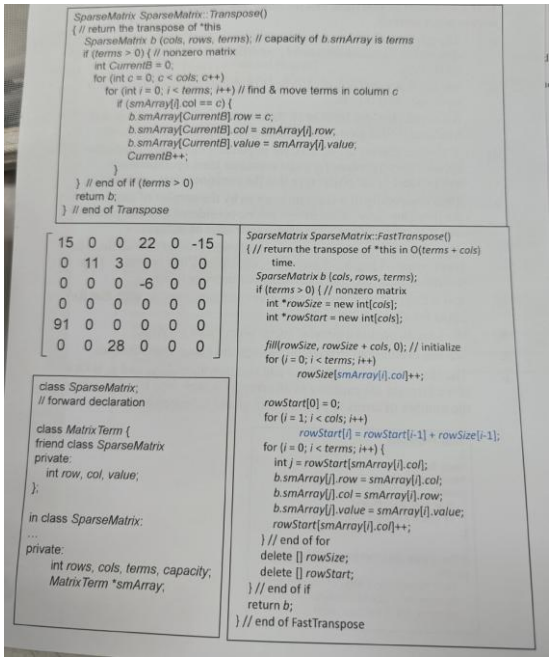
若陣列每次滿了只增加 3 格容量，則將兩個多項式 (p_1) 與 (p_2) 相加的時間複雜度為 (O(m+n))。

```
class Polynomial;

class Term {
friend Polynomial;
private:
   float coef; // coefficient
   int exp; // exponent
};

// The private data members of Polynomial
private:
   Term *termArray; // array of nonzero terms
   int capacity; // size of termArray
   int terms; // # of nonzero terms
```

j. False

這題很容易掉進陷阱。若兩個多項式相加，本身合併項目的邏輯確實像 merge，一般是 $O(m+n)$。但是題目特別補了一句：**陣列每次滿了只增加 3 格容量**。

這代表擴充不是倍增，而是固定增加常數。若結果項數很多，會頻繁重新配置並複製陣列內容。這樣總成本不再是線性，可能累積成平方等級。也就是說，雖然單純邏輯上的合併是 $O(m+n)$，但把「固定只加 3 格的擴充成本」算進去後，整體就不能保證仍是 $O(m+n)$。所以這題是 **False**。



```
SparseMatrix SparseMatrix::Transpose()
{ // return the transpose of *this
  SparseMatrix b (cols, rows, terms); // capacity of b.smArray is terms
  if (terms > 0) { // nonzero matrix
    int CurrentB = 0;
    for (int c = 0; c < cols; c++)
      for (int i = 0; i < terms; i++) // find & move terms in column c
        if (smArray[i].col == c) {
          b.smArray[CurrentB].row = c;
          b.smArray[CurrentB].col = smArray[i].row;
          b.smArray[CurrentB].value = smArray[i].value;
          CurrentB++;
        }
  } // end of if (terms > 0)
  return b;
} // end of Transpose
```

15	0	0	22	0	-15
0	11	3	0	0	0
0	0	0	-6	0	0
0	0	0	0	0	0
91	0	0	0	0	0
0	0	28	0	0	0

```
class SparseMatrix;
// forward declaration

class MatrixTerm {
friend class SparseMatrix
private:
   int row, col, value;
};

in class SparseMatrix:
...
private:
      int rows, cols, terms, capacity;
      MatrixTerm *smArray;
```

```
SparseMatrix SparseMatrix::FastTranspose()
{ // return the transpose of *this in O(terms + cols)
      time.
  SparseMatrix b (cols, rows, terms);
  if (terms > 0) { // nonzero matrix
    int *rowSize = new int[cols];
    int *rowStart = new int[cols];

    fill(rowSize, rowSize + cols, 0); // initialize
    for (i = 0; i < terms; i++)
          rowSize[smArray[i].col]++;

    rowStart[0] = 0;
    for (i = 1; i < cols; i++)
          rowStart[i] = rowStart[i-1] + rowSize[i-1];
    for (i = 0; i < terms; i++) {
      int j = rowStart[smArray[i].col];
      b.smArray[j].row = smArray[i].col;
      b.smArray[j].col = smArray[i].row;
      b.smArray[j].value = smArray[i].value;
      rowStart[smArray[i].col]++;
    } // end of for
    delete [] rowSize;
    delete [] rowStart;
  } // end of if
  return b;
} // end of FastTranspose
```

以下是第 2 題（Sparse Matrix）的 **英文題目＋中文翻譯＋完整詳解（含答案）**，依你要求整合在一起：

第 2 題英文題目

2. SparseMatrix (15 pts)

a. Please show the result of the following matrix when is stored by an array of triples <row, col, value>.

b. What is the time complexity of Transpose?

c. What is the time complexity of FastTranspose?

第 2 題中文翻譯

2. 稀疏矩陣（15 分）

a. 請將下列矩陣以三元組（<row, col, value>）的陣列形式表示出來。

b. Transpose（一般轉置）的時間複雜度為何？

c. FastTranspose（快速轉置）的時間複雜度為何？

詳細解答（答案整合）

(a) 三元組表示法

先觀察矩陣（6×6）：

15	0	0	22	0	-15
0	11	3	0	0	0
0	0	0	-6	0	0
0	0	0	0	0	0
91	0	0	0	0	0
0	0	28	0	0	0

非零元素依序為（以 row, col, value 表示）：

(0,0,15)
(0,3,22)
(0,5,-15)
(1,1,11)
(1,2,3)
(2,3,-6)
(4,0,91)
(5,2,28)

通常 sparse matrix 的 triple array 第一列會放「矩陣資訊」：

(rows, cols, terms) = (6, 6, 8)

因此完整三元組陣列為：

(6,6,8)
(0,0,15)
(0,3,22)
(0,5,-15)
(1,1,11)
(1,2,3)
(2,3,-6)

(4,0,91)
(5,2,28)

(b) Transpose 時間複雜度

根據題目給的程式碼：

```
for (c = 0 → cols-1)
    for (i = 0 → terms-1)
```

這是**雙層迴圈**：

- 外層跑 cols 次
- 內層跑 terms 次

因此時間複雜度為：

O(cols × terms)

☞ **答案：O(cols × terms)**

(c) FastTranspose 時間複雜度

FastTranspose 做了三件事：

1. 計算每一 column 有幾個元素 → O(terms)
2. 計算 rowStart → O(cols)
3. 再掃一次資料放入正確位置 → O(terms)

總時間：

O(terms + cols)

☞ **答案：O(terms + cols)**

總結（考試寫法）

(a)
(6,6,8)
(0,0,15)
(0,3,22)
(0,5,-15)
(1,1,11)
(1,2,3)
(2,3,-6)
(4,0,91)
(5,2,28)

(b)
O(cols × terms)

(c)
O(terms + cols)

.英文題目

3. (20 pts) Please compute the time complexity of SelectionSort Algorithm according to the three methods in the following (assume it takes O(1) to swap two elements (i.e., swap(a[i], a[j])):

```
C++

void SelectionSort (int *a, const int n)
{// Sort the n integers a[0] to a[n-1] into nondecreasing order
1.  for (int i = 0; i < n; i++)
2.  {
3.      int j = i;
4.      // find smallest integer in a[i] to a[n-1]
5.      for (int k = i + 1; k < n; k++)
6.          if (a[k] < a[j])
7.              j = k;
8.      swap(a[i], a[j]);
9.  }
}
```

a. **(10 pts) Please insert a new variable, count, into the program and show a simplified version of the original program?**

b. **(10 pts) Please build a table which lists the total number of steps contributed by each statement as follows:**

3.（20 分）請依照下列三種方法來計算 SelectionSort（選擇排序）演算法的時間複雜度。假設交換兩個元素需要的時間是 O(1)，也就是 swap(a[i], a[j]) 為常數時間。

a. （10 分）請在程式中加入一個新變數 count，並寫出原程式的簡化版本。

b. （10 分）請建立一個表格，列出每一行敘述所貢獻的總步數，表格欄位為：line、s/e、frequency、total steps。

3.詳細解答1.

這一題是在考你怎麼把 **Selection Sort 的程式執行次數** 寫成數學式，再進一步求出時間複雜度。Selection Sort 的核心概念是：第 i 回合時，先假設第 i 個位置就是目前最小值的位置 $j = i$，再從後面區間 $a[i+1] \sim a[n-1]$ 去找更小的元素。如果找到，就更新 j；最後把 $a[i]$ 與 $a[j]$ 交換。這件事會做 n 輪，因此外層迴圈是主要架構；而真正耗時間最多的是內層搜尋最小值的比較動作。 2024資料結構 midterm 1

先回答 (a) 加入 `count` 的簡化程式。所謂 simplified version，就是把原本每一行的執行都視為某個固定成本，並在執行時累加到 `count`。可以寫成下面這種形式：

```
void SelectionSort(int *a, const int n)
{
    int count = 0;
    for (int i = 0; i < n; i++) {       // line 1
```

```
        count++;                        // 外層迴圈本體進入一次
        int j = i;                      // line 3
        count++;
        for (int k = i + 1; k < n; k++) {  // line 5
            count++;
            if (a[k] < a[j]) {          // line 6
                count++;
                j = k;                  // line 7
                count++;
            }
        }
        swap(a[i], a[j]);               // line 8
        count++;
    }
}
```

若老師要更數學化的版本，也可以直接把重點濃縮成：

- line 3 會做 n 次
- line 5, 6 的核心比較總共做 $(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$ 次
- line 8 會做 n 次
  所以整體主導項來自內層比較，最後是二次時間。 2024資料結構 midterm 1

接著回答 (b) 的表格。這裡常見的寫法是令每一行單次執行成本分別為 $c_1, c_2, \ldots, c_9$。由於第 2 行與第 9 行只是大括號，通常不算實際執行步驟，或可視為 0。真正重要的是 line 1、3、5、6、7、8。依 Selection Sort 的結構可得：

- line 1 外層 `for (int i = 0; i < n; i++)` 的判斷大約做 $n+1$ 次
- line 3 `int j = i;` 做 n 次
- line 5 內層 `for (int k = i+1; k < n; k++)` 的判斷總次數為

$$\sum_{i=0}^{n-1}(n-i)=n+(n-1)+\cdots+1=\frac{n(n+1)}{2}$$

  若只算迴圈本體執行次數則是

$$\sum_{i=0}^{n-1}(n-i-1)=\frac{n(n-1)}{2}$$

- line 6 比較 `if (a[k] < a[j])` 的次數為

$$\frac{n(n-1)}{2}$$

- line 7 `j = k` 的次數不固定，因為要看輸入資料。最好情況可能 0 次，最壞情況可到

$$\frac{n(n-1)}{2}$$

  但不管怎樣都不會改變最終階數
- line 8 `swap(a[i], a[j])` 做 n 次

因此表格可寫成：

因此表格可寫成：

line	s/e	frequency	total steps
1	c_1	$n+1$	$c_1(n+1)$
2	0	0	0
3	c_3	n	$c_3 n$
4	0	0	0
5	c_5	$\frac{n(n+1)}{2}$ 或本體 $\frac{n(n-1)}{2}$	對應乘上 c_5
6	c_6	$\frac{n(n-1)}{2}$	$c_6\frac{n(n-1)}{2}$
7	c_7	視資料而定，最壞 $\frac{n(n-1)}{2}$	最壞 $c_7\frac{n(n-1)}{2}$
8	c_8	n	$c_8 n$
9	0	0	0

1.英文題目

3. (20 pts) Please compute the time complexity of SelectionSort Algorithm according to the following methods (assume swap takes O(1)).

```
void SelectionSort (int *a, const int n)
{// Sort the n integers a[0] to a[n-1] into nondecreasing order
  for (int i = 0; i < n; i++)
  {
      int j = i;
      // find smallest integer in a[i] to a[n-1]
      for (int k = i + 1; k < n; k++)
          if (a[k] < a[j])
              j = k;
      swap(a[i], a[j]);
  }
}
```

2.題目中文翻譯

3.（20 分）請計算 Selection Sort（選擇排序）演算法的時間複雜度（假設交換操作為 O(1)）。

3.詳細解答 1.

(a) 加入 count 的簡化程式

```
void SelectionSort(int *a, const int n)
{
    int count = 0;
    for (int i = 0; i < n; i++) {
        count++;
        int j = i;
        count++;
        for (int k = i + 1; k < n; k++) {
            count++;
            if (a[k] < a[j]) {
                count++;
                j = k;
                count++;
            }
        }
        swap(a[i], a[j]);
        count++;
    }
```

}

(b) 各行步數分析表

line 1：frequency = $n+1$

line 3：frequency = n

line 5：frequency = $n(n-1)/2$

line 6：frequency = $n(n-1)/2$

line 7：frequency $\leq n(n-1)/2$（最壞）

line 8：frequency = n

Total steps：

$T(n) = c_1(n+1) + c_3 n + c_5 \cdot n(n-1)/2 + c_6 \cdot n(n-1)/2 + c_7 \cdot (\leq n(n-1)/2) + c_8 n$

整理後主導項為

$T(n) = an^2 + bn + c$

因此時間複雜度為

$O(n^2)$（同時也是 $\Theta(n^2)$）

補充說明（考試加分寫法）：

Selection Sort 在每一輪都需要掃描剩餘未排序元素來找最小值，因此比較次數固定為 $n(n-1)/2$，不會因輸入資料不同而改變，所以最佳、平均與最壞情況皆為 $O(n^2)$。

1.英文題目

4. (10 pts) Give a table which characterizes the priority of the operators. The expression is (A + !B) ll (C*D) % E. Please show the status of the stack and output when you transform the infix expression to the postfix expression in C++ by extracting a token from the equation in serial.

2.題目中文翻譯

4.（10 分）請根據題目所給的運算子優先順序表，將中序運算式

(A + !B)\ ll\ (C*D)%E

轉換成後序運算式（postfix expression）。並請依照 C++ 程式中逐一讀取 token 的方式，寫出轉換過程中 stack 與 output 的狀態變化。

3.詳細解答

這一題是在考 infix 轉 postfix，也就是把一般人習慣看的中序表示法，轉成電腦較方便處理的後序表示法。題目右上角已給出優先順序表：！ 最高，其次是 * / %，再來是 + -，最後 ||。因此本題運算式

(A + !B)\ ll\ (C*D)%E

要先算左邊括號中的 !B，再做 A + (!B)；右邊括號中先做 C*D，再把結果 % E；最後再把左右兩部分以 || 相連。由於 postfix 的原則是「運算元先輸出，運算子延後輸出」，所以我們在掃描 token 時，若遇到 operand（如 A、B、C、D、E）就直接放到 output；若遇到運算子，就依照優先順序與 stack top 比較，決定 push 或 pop；若遇到右括號)，就一直 pop 直到碰到左括號 (為止。

本題 token 順序為：

(、A、+、!、B、)、||、(、C、*、D、)、%、E

題目程式還說 stack 底部先放 #，最後 expression 結束時再把 stack 清空。因此我們逐步模擬如下：

讀入 token	動作說明	Stack 狀態	Output 狀態
初始	push #	#	空
(	左括號直接 push	# (	
A	operand，直接輸出	# (	A
+	push +	# (+	A
!	！ 優先權高於 +，直接 push	# (+ !	A
B	operand，直接輸出	# (+ !	AB
)	pop 到 (，先輸出 ！，再輸出 +	#	AB!+

`		`	push `
(	左括號直接 push	`#	
C	operand，直接輸出	`#	
*	push *	`#	
D	operand，直接輸出	`#	
)	pop 到 (，輸出 *	`#	
%	% 優先權高於 `		`，直接 push
E	operand，直接輸出	`#	
結束	清空 stack，先 pop % 再 pop `		`

所以最後得到的 postfix expression 為：

AB!+CD*E%||

若寫成連續字串，就是：

AB!+CD*E%||

這個答案也可以從語意上再次驗證：左半部 (A + !B) 轉成 postfix 是 AB!+，因為先輸出 A、B，再做 !，最後做 +；右半部 (C*D)%E 轉成 postfix 是 CD*E%，因為先做 C*D，再把結果與 E 做 %；最後整個式子是左半部與右半部做 ||，所以把 || 放在最後，得到 AB!+CD*E%||。這完全符合 postfix 的規則，也符合題目提供的優先順序表，因此這就是本題標準答案。

1.英文題目

5. (5 pts) String Pattern Matching

$$f(j) = \begin{cases} -1, & \text{if } j = 0; \\ f^m(j-1)+1, & \text{where } m \text{ is the least int } k \text{ s.t. } p_{f^k(j-1)+1} = p_j; \\ -1, & \text{otherwise.} \end{cases}$$

Note: $f^{-1}(j) = j,\ f^m(j) = f(f^{m-1}(j))$

Complete the failure function of the pattern shown in the following.

題目所給 pattern 為：

$a\ b\ a\ c\ a\ b\ a\ c\ a\ b$ 2024資料結構 midterm 1

2.題目中文翻譯

5.（5 分）字串樣式比對註： (f^{-1}(j)=j,; f^m(j)=f(f^{m-1}(j)))

請完成下列 pattern 的 failure function。

題目給的樣式字串是：

abacabacab

3.詳細解答

這一題是在考 KMP 的 failure function。它的意思是：當我們比對到 pattern 的第 (j) 個字元失敗時，不需要整個回到開頭，而是可以跳到某個適當的位置，繼續利用先前已經知道的「前綴 = 後綴」資訊。題目已經先給出 (f(0)=-1)，這代表第一個字元若失敗，沒有任何可回跳的位置，所以記成 -1。之後每一個 (f(j)) 的值，其實就是在看：對於子字串 (P[0..j])，它最長的 proper prefix 與 proper suffix 相同時，那個對應位置的最後索引是多少。

本題 pattern 是 abacabacab，我們逐一來看。

當 (j=0) 時，子字串是 a，題目已給 (f(0)=-1)。

當 (j=1) 時，子字串是 ab，proper prefix 有 ""、a，proper suffix 有 ""、b，沒有非空相同，所以 (f(1)=-1)。

當 (j=2) 時，子字串是 aba，proper prefix 有 a、ab，proper suffix 有 a、ba，最長相同的是 a，長度 1，因此最後索引是 0，所以 (f(2)=0)。

當 (j=3) 時，子字串是 abac，prefix 與 suffix 沒有非空相同，所以 (f(3)=-1)。

當 (j=4) 時，子字串是 abaca，最長相同 prefix/suffix 是 a，所以 (f(4)=0)。

當 (j=5) 時，子字串是 abacab，最長相同 prefix/suffix 是 ab，最後索引是 1，所以 (f(5)=1)。

當 (j=6) 時，子字串是 abacaba，最長相同 prefix/suffix 是 aba，最後索引是 2，所以 (f(6)=2)。

當 (j=7) 時，子字串是 abacabac，最長相同 prefix/suffix 是 abac，最後索引是 3，所以 (f(7)=3)。

當 (j=8) 時，子字串是 abacabaca，最長相同 prefix/suffix 是 abaca，最後索引是 4，所以 (f(8)=4)。

當 (j=9) 時，子字串是 abacabab? 不對，依題目 pattern 應為 abacabacab 的前十個字元，也就是 abacabacab，最長相同 prefix/suffix 是 abacab，最後索引是 5，所以 (f(9)=5)。

因此整個 failure function 應為：
(f(0)=-1)
(f(1)=-1)
(f(2)=0)
(f(3)=-1)
(f(4)=0)
(f(5)=1)
(f(6)=2)
(f(7)=3)
(f(8)=4)
(f(9)=5)

如果用表格寫成考卷答案會更清楚：

$$
\begin{array}{l}
j: \quad 0\ 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9 \\
P[j]: \ a\ b\ a\ c\ a\ b\ a\ c\ a\ b \\
f(j): \ -1\ -1\ 0\ -1\ 0\ 1\ 2\ 3\ 4\ 5
\end{array}
$$

所以本題最後答案就是：
failure function = $[-1,\ -1,\ 0,\ -1,\ 0,\ 1,\ 2,\ 3,\ 4,\ 5]$

這個結果也很合理，因為這個 pattern abacabacab 本身具有明顯重複結構：前半段 abac 幾乎又在後面重現一次，所以從 (j=4) 之後 failure 值會逐漸往上增加，表示當後面比對失敗時，可以回跳到前面已知相同結構的位置，而不用完全重頭開始。這正是 KMP 能比 naive matching 更快的原因。

補充（加分寫法）：
failure function 表示當第 j 個字元比對失敗時，pattern 應跳回的位置，其本質為「最長相同的 proper prefix = proper suffix」。本題 pattern 具有重複結構 abac，因此從 j=4 開始 failure 值逐漸增加，可有效避免重複比對，這也是 KMP 演算法能達到 O(n+m) 的關鍵。

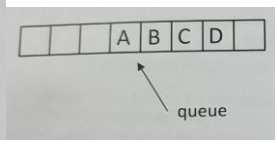
1. 英文題目

6. (10 pts) Circular queue

a. (5 pts) Please show the status of the circular queue (including the locations of front and rear) after the following three operations are applied? Push(E), Pop(), Push(F)

b. (5 pts) According to the result of (a), please show the status of the circular queue (including the locations of front and rear) after the following three operations are applied (hint: you can reference the following code)? Push(G), Push(H), Push(I)

```
//allocate an array with twice the capacity
1. T* newQueue = new T [2*capacity];
// 從queue複製至 newQueue
2. int start = (front + 1) % capacity;
3. if (start < 2) //front points to the first and last position in the queue
   // no wrap around is required
4.    copy (queue + start, queue + start + capacity – 1, newQueue);
5. else
6. {    // queue wraps around
7.      copy (queue + start, queue + capacity, newQueue);
8.      copy (queue, queue + rear + 1, newQueue + capcity – start);
9. }
// 切換至 newQueue
10. front = 2 * capacity – 1; // point to the last location in the new queue
11. rear = capacity – 2;
12. capacity *= 2;
13. delete [] queue;
14. queue = newQueue;
```



2.題目中文翻譯

6.（10 分）環狀佇列（Circular queue）

a.（5 分）請在依序執行 Push(E)、Pop()、Push(F) 三個操作之後，畫出 circular queue 的狀態，並標示 front 與 rear 的位置。

b.（5 分）根據 (a) 的結果，再依序執行 Push(G)、Push(H)、Push(I) 三個操作。請畫出 circular queue 的狀態，並標示 front 與 rear 的位置。（提示：可參考右側給的 doubling code）

3.詳細解答

這一題是在考 circular queue 的移動方式，以及當 queue 滿掉時，如何根據題目右邊提供的程式碼去做 容量加倍（doubling）。先看題目一開始給的圖，整個 queue 有 8 格，內容依序是：第 3、4、5、6 格放著 A B C D，其餘位置是空的。由圖的畫法可知目前 queue 中的資料順序是 A → B → C → D，而在 circular queue 的常見表示法中，front 通常指向「第一個元素前一格」，rear 指向「最後一個元素所在位置」。因此初始狀態可視為：

front = 2，rear = 6

因為真正的第一個元素 A 在 (front+1) % 8 = 3 的位置，而最後一個元素 D 在 6。

先做 (a)。
第一步 Push(E)：把 E 放到 rear 的下一格，也就是位置 7，所以此時 queue 變成
位置 3=A，4=B，5=C，6=D，7=E
因此 rear = 7，front 不變仍是 2。
第二步 Pop()：刪除最前面的元素，也就是 A。由於 front 指向第一個元素前一格，所以 pop 後 front 往前移一格，變成 3。這時真正的第一個元素變成 B（在位置 4）。queue 內容變成
位置 4=B，5=C，6=D，7=E
因此此時 front = 3，rear = 7。
第三步 Push(F)：把 F 放到 rear 的下一格，因為是 circular queue，所以 (7+1) % 8 = 0，因此 F 會放到位置 0，形成環回。此時 queue 內容依 queue 順序為
B → C → D → E → F
實際陣列位置是
0=F，4=B，5=C，6=D，7=E
所以 (a) 做完後：
front = 3，rear = 0。

(a) 的答案可直接寫成：
位置 0 1 2 3 4 5 6 7
內容 F _ _ _ B C D E
front = 3，rear = 0
queue 順序：B, C, D, E, F

接著做 (b)，要從 (a) 的結果繼續。
目前 capacity = 8，front = 3，rear = 0。
此時 queue 中共有 5 個元素：B、C、D、E、F。

第一步 Push(G)：放到 (rear+1)%8 = 1，所以 G 在位置 1，rear = 1。
第二步 Push(H)：放到 (1+1)%8 = 2，所以 H 在位置 2，rear = 2。
做完這兩步後，queue 內有 7 個元素，這在保留一格空位的 circular queue 中代表「已滿」。目前狀態是：
0=F，1=G，2=H，4=B，5=C，6=D，7=E，且 front = 3，rear = 2。
第三步要 Push(I) 時會發現 queue 已滿，因此要先 doubling。依題目右邊程式碼：
start = (front + 1) % capacity = (3+1)%8 = 4
因為 start = 4，不小於 2，所以屬於 wrap around 的情況。
因此會先複製 queue[4] 到 queue[7]，也就是 B C D E 到新陣列前面；

再複製 queue[0] 到 queue[rear]，也就是 F G H 接在後面。
所以複製到新陣列（大小 16）後，前 7 格會依序變成：
0=B，1=C，2=D，3=E，4=F，5=G，6=H
接著根據題目程式碼：
front = 2*capacity - 1 = 15
rear = capacity - 2 = 6
然後 capacity = 16。
最後再執行真正的 Push(I)，把 I 放到 (rear+1)%16 = 7，因此 I 進入位置 7，rear = 7。

所以 (b) 最後答案為：
新容量 capacity = 16
位置 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
內容 B C D E F G H I _ _ _ _ _ _ _ _
front = 15，rear = 7
queue 順序：B, C, D, E, F, G, H, I

因此本題完整答案可整理如下：
(a) 執行 Push(E), Pop(), Push(F) 後
位置 0 1 2 3 4 5 6 7
內容 F _ _ _ B C D E
front = 3，rear = 0

(b) 再執行 Push(G), Push(H), Push(I) 後，因為第三步發生滿佇列，所以先 doubling 再插入 I，最後得到
位置 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
內容 B C D E F G H I _ _ _ _ _ _ _ _
front = 15，rear = 7

這題最重要的觀念有兩個：第一，**front 指向第一個元素前一格**，所以 pop 時是 front 往前移；第二，**當 circular queue 滿時且發生 wrap around，doubling 要先從 start 到尾端複製，再從開頭複製到 rear**，這樣才能保持元素的邏輯順序不亂。只要抓住這兩點，這題就很好拿分。

1.英文題目

7. (5 pts) How many space is required when you store a polynomial equation

$f(x)=3x^{100}+4x^{98}+50x^{30}+49$ by the representation 1 and the representation 2, respectively?

Representation 1

```
private:
    int degree;
    float *coef;

Polynomial::Polynomial(int d)
{
        degree = d;
        coef = new float [degree+1];
}
```

Representation 2

```
class Polynomial;

class Term {
friend Polynomial;
private:
    float coef; // coefficient
    int exp; // exponent
};
Polynomial {
// The private data members of Polynomial
private:
    Term *termArray; // array of nonzero terms
    int capacity; // size of termArray
    int terms; // # of nonzero terms
```

2.題目中文翻譯

7.（5 分）當你使用 Representation 1 與 Representation 2 來儲存多項式

$f(x)=3x^{100}+4x^{98}+50x^{30}+49$ 時，分別需要多少空間？

3.詳細解答

這題是在比較「稠密表示（array by degree）」與「稀疏表示（只存非零項）」的空間使用差異。先看 Representation 1，其資料結構包含一個 int degree，以及一個 float *coef，而 coef 會配置大小為 degree + 1 的陣列。題目中最高次項為 x^{100}，因此 degree = 100，所以需要配置 101 個 float 空間來存放從 x^{0} 到 x^{100} 的所有係數。即使中間大多數係數都是 0（例如 (x^{99}, x^{97} …)），仍然必須全部配置，因此空間使用為

101 個 float + 1 個 int。如果只用「元素數量」來回答，就是 **101 個係數空間**。這種方法的特點是存取方便，但對於稀疏多項式會浪費大量空間。

再看 Representation 2，它只存「非零項」，每一項用一個 Term 結構表示，包含 float coef 與 int exp。題目中的多項式共有 4 個非零項：

$3x^{100}+4x^{98}+50x^{30}+49$，因此只需要 4 個 Term。每個 Term 需要 1 個 float + 1 個 int，所以總共需要

4 × (1 float + 1 int)。此外 Polynomial 類別本身還有 Term* termArray、int

capacity、int terms 三個成員，但在考試中通常重點是 termArray 所需空間，因此主要答案為 4 個非零項的儲存空間。

因此整理答案：
Representation 1：需要 **101 個 float（對應 degree+1）**
Representation 2：需要 **4 個 Term（每個包含 coef 與 exp）**

補充說明（加分）：
Representation 1 屬於稠密表示法，適合多數係數非零的情況；Representation 2 屬於稀疏表示法，當多項式只有少數非零項時能大幅節省空間。本題中最高次為 100，但只有 4 項非零，因此使用 Representation 2 會遠比 Representation 1 節省空間，這正是資料結構設計中常見的空間效率比較重點。

1.英文題目

8. A Mazing Problem (20 pts)

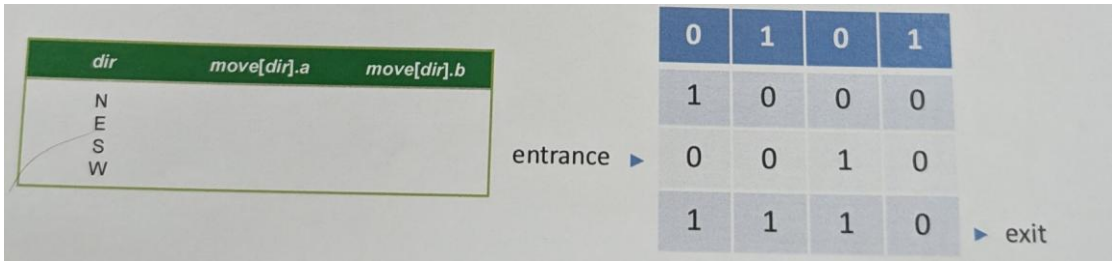
The rat in a maze experiment is a classical one from experimental psychology, where a map is recorded by a two-dimensional array, maze[i][j], where (1 \le i \le m) and (1 \le j \le p). A value of 1 implies a blocked path, and a 0 means one can walk right on through. Assume you can go to the four directions (north, east, south, and west) only. The data types needed are

```
struct offsets { int a, b; };
enum directions {N, E, S, W};
offsets move[4];
```

```
initialize list to the maze entrance coordinates and direction north;
while (list is not empty)
{
    (i, j, dir) = coordinates and direction from end of list;
    delete last element of list;
    while (there are more moves from (i, j)) // i.e., dir < 4
    {
        (g, h) = coordinates of next move;
        if ((g == m) && (h == p)) success;
        if (!maze[g][h])
        {
            dir = next direction to try;
            add (i, j, dir) to end of list;
            (i, j, dir) = (g, h, N); // move to new location
        }
    }
}
cout << "No path in maze." << endl;
```

a. (3 pts) What kind of the data structure that you can use to implement the list in the above pseudocode? Why?

b. (6 pts) If you are at the location [i][j], and you want to go the direction dir, please complete the following table and show the equation to get the coordinate of a new location [g][h].



c. (4 pts) Please modify the above code to ensure that the rat does not go the same location repeatedly (you only need to highlight the codes that you add in the correct locations according to the above pseudo codes).

d. (5 pts) Please show the path that is found by the program in the above map.

e. (2 pts) Can this algorithm always find a path if there exists at least one route from entrance to exit according to the algorithm?

2.題目中文翻譯

8. 迷宮問題（20 分）

老鼠走迷宮實驗是實驗心理學中的經典問題之一。地圖以二維陣列 maze[i][j] 表示，其中 (1 \le i \le m)、(1 \le j \le p)。若陣列值為 1，表示此處是阻塞、不可通行；若值為 0，表示可以通行。假設老鼠只能朝四個方向移動：北（N）、東（E）、南（S）、西（W）。所需資料型別如下：

```
struct offsets { int a, b; };
enum directions {N, E, S, W};
offsets move[4];
```

1. 將 list 初始化為迷宮入口座標，方向設為北方；
2. 當 list 不為空時；
3. {
    4. 從 list 尾端取出座標與方向 ((i, j, dir))；
    5. 刪除 list 最後一個元素；
    6. 當從 ((i, j)) 還有可嘗試的移動方向時（也就是 dir < 4）；
    7. {
    8. ((g, h)) = 下一步移動後的新座標；
    9. 如果 ((g == m)) 且 ((h == p))，表示成功到達出口；
    10. 如果 maze[g][h] == 0，表示可走；
    11. {
    12. dir = 下一個要嘗試的方向；
    13. 將 ((i, j, dir)) 加到 list 尾端；
    14. 令 ((i, j, dir) = (g, h, N))`，表示移動到新位置並重新從北開始嘗試；

```
        15.     }
        16.   }
    4.  }
    5.  若最後仍找不到路，輸出 "No path in maze."
```

a. 上述虛擬碼中的 list 應該使用哪一種資料結構來實作？為什麼？
b. 若目前位於 [i][j]，要朝方向 dir 移動，請完成表格，並寫出新位置 [g][h] 的計算公式。
c. 請修改上述程式，使老鼠不會重複走到相同位置。（只需標出你新增的程式碼及其應放的位置）
d. 請寫出此程式在題目迷宮中所找到的路徑。
e. 根據此演算法，只要入口到出口至少存在一條路，它是否一定能找到？

3.詳細解答
這一題其實是在考 **stack 實作的迷宮深度優先搜尋（DFS）**。因為虛擬碼中反覆使用「從 list 的尾端取出資料」以及「把新狀態加到 list 尾端」，這代表它採用的是 **後進先出 LIFO** 的行為，因此 (a) 應使用 **stack**。原因是目前位置若還能往更深處走，就先把目前狀態存起來，繼續往前探索；若走不通，再從最近一次儲存的位置回溯，這正是 stack 最典型的用途。

(b) 題目給的方向順序是 N, E, S, W。若以二維陣列常見座標定義來看，往北代表 row 減 1、col 不變；往東代表 row 不變、col 加 1；往南代表 row 加 1、col 不變；往西代表 row 不變、col 減 1。因此表格應為：

```
N : move[N].a = -1, move[N].b = 0
E : move[E].a = 0, move[E].b = 1
S : move[S].a = 1, move[S].b = 0
W : move[W].a = 0, move[W].b = -1
```

新座標公式是：

$$g=i+move[dir].a, \qquad h=j+move[dir].b$$

(c) 為了避免老鼠重複走同一個位置，需要加入 **visited / mark 陣列**，或直接把走過的位置標記起來。最簡單做法是在成功走進新位置時立刻標記。例如可新增：
`bool visited[m+1][p+1] = {false};`
初始化入口時：`visited[entrance_i][entrance_j] = true;`
並把原本第 10 行條件改成：
`if (!maze[g][h] && !visited[g][h])`

進入新位置時再加：
visited[g][h] = true;
這樣每個位置最多只會被走一次，就不會在迷宮裡反覆繞圈。

(d) 再看題目右下角的迷宮。由圖可讀成 4×4：
第 1 列：0 1 0 1
第 2 列：1 0 0 0
第 3 列：0 0 1 0
第 4 列：1 1 1 0

入口在左邊指向第 3 列第 1 欄，也就是 **(3,1)**；出口在右邊指向第 4 列第 4 欄，也就是 **(4,4)**。依題目演算法方向順序 N→E→S→W 來找：
從 (3,1) 出發，先試 N 到 (2,1)，但這格是 1，不可走；
試 E 到 (3,2)，可走；
到 (3,2) 後試 N 到 (2,2)，可走；
到 (2,2) 後試 N 到 (1,2) 是 1，不行；
試 E 到 (2,3)，可走；
到 (2,3) 後試 N 到 (1,3)，可走；
到 (1,3) 後四周都無法通往出口，會回溯；
回到 (2,3) 再試 E 到 (2,4)，可走；
到 (2,4) 後試 N 到 (1,4) 是 1，不行；
試 E 越界，不行；
試 S 到 (3,4)，可走；
到 (3,4) 後試 S 到 (4,4)，到達出口。

所以找到的路徑可寫成：
(3,1)→(3,2)→(2,2)→(2,3)→(2,4)→(3,4)→(4,4)
如果把曾經探索但最後回溯的點也寫進過程，則會包含 (1,3)，但「最終找到的 path」通常寫成功路徑即可。

(e) 這個演算法在**有記錄 visited、避免重複走同一位置**的前提下，只要迷宮有限大，而且入口到出口確實存在至少一條可通行路徑，它就 **一定找得到**。原因是這其實就是 DFS：它會系統性地把每個可能位置依序展開，走不通就回溯，只要所有可達位置都被完整探索，存在的路徑不會被漏掉。若沒有 visited 機制，則可能在某些迷宮中反覆繞圈，導致不保證結束，也就不能保證一定找到。所以本題最完整的答案是：**若加上避免重複拜訪的位置標記，則可以；若不加，則不一定。**

綜合整理：
(a) list 用 stack。
(b) N=(-1,0), E=(0,1), S=(1,0), W=(0,-1)，公式為 g=i+move[dir].a, h=j+move[dir].b。
(c) 加入 visited 檢查與標記。
(d) 路徑為 (3,1)→(3,2)→(2,2)→(2,3)→(2,4)→(3,4)→(4,4)。
(e) 加上 visited 時，此演算法可以找到存在的路徑；不加則不一定。

如果你要，我下一則可以直接幫你整理成 **第 8 題考卷可直接抄寫版**。

1.英文題目
8. A Mazing Problem (20 pts)
(題目略，為以二維陣列表示迷宮並用 list 搜尋路徑之問題)

2.題目中文翻譯
8. 迷宮問題（20 分），使用 list 模擬老鼠在迷宮中找路，並回答資料結構、移動公式、程式修改、路徑與正確性。

3.詳細解答
(a)
使用資料結構：**Stack（堆疊）**
理由：程式中使用「從 list 尾端取出元素」與「加入到尾端」，屬於 LIFO（後進先出），即 DFS（深度優先搜尋），因此需用 stack。

(b)
方向與位移：
N：(-1, 0)
E：(0, 1)
S：(1, 0)
W：(0, -1)

新座標公式：
g = i + move[dir].a，h = j + move[dir].b

(c)
避免重複走同一位置（加入 visited）：
在宣告區加入：
```
bool visited[m+1][p+1] = {false};
```
初始化入口：
```
visited[i][j] = true;
```

修改條件（line 10）：

```
if (!maze[g][h] && !visited[g][h])
```

進入新位置時加入：

```
visited[g][h] = true;
```

(d)

迷宮路徑：

(3,1) → (3,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4)

(e)

是否一定找到路徑：

會（若有 visited），因為此演算法為 DFS，會完整探索所有可行路徑；

不一定（若無 visited），可能陷入重複循環。

最終重點（加分一句）：

本題本質為 **DFS + stack + backtracking**，搭配 visited 可保證找到解。